

MODELING USER PERSPECTIVE IN RUMOUR DISSEMINATION

A Project Progress Report

Submitted in partial fulfillment for the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING

Submitted By

P LEELA

(N190456)

Under the Esteem Guidance of

Mrs. M.J. Blessy

Assistant Professor in Department of Computer Science & Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru(Dist), Andhra Pradesh - 521202



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru(Dist), Andhra Pradesh – 521202

CERTIFICATE OF COMPLETION

This is to certify that the work entitled, “**MODELING USER PERSPECTIVE IN RUMOUR DISSEMINATION**” is the bonafide work of **P. LEELA (ID NO.: N190456)** carried out under my guidance and supervision for 4th year project of **Bachelor of Technology** in the department of Computer Science and Engineering under RGUKT IIIT Nuzvid.

Mrs. M.J. Blessy

Assistant Professor,

Department of CSE

RGUKT- Nuzvid

Mrs. S. Bhavani

Assistant Professor,

Head of the Department CSE,

RGUKT- Nuzvid



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru(Dist), Andhra Pradesh – 521202

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled, “**MODELING USER PERSPECTIVE IN RUMOUR DISSEMINATION**” is the bonafide work of **P. LEELA (ID NO.: N190456)** and hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in the 4th year of **Bachelor of Technology** for which it has been submitted. This approval does not endorse or accept every statement made, opinion expressed, or conclusion drawn, as recorded in this report for the purpose for which it has been submitted.

Mrs. M. J. Blessy

Assistant Professor,

Department of CSE,

RGUKT- Nuzvid.

PROJECT EXAMINER

Department of CSE

RGUKT- Nuzvid.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru(Dist), Andhra Pradesh – 521202

DECLARATION

I **“P. LEELA (ID NO.: N190456)”** hereby declare that the project report entitled **“MODELING USER PERSPECTIVE IN ROMOUR DISSEMINATION ”** done by us under the guidance of Mrs. M. J.Blessy, Assistant Professor is submitted for the partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites are mentioned in the references. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

Date : 26 -04-2025

Place: Nuzvid

P. LEELA (N190456)

ACKNOWLEDGEMENT

We would like to express our profound gratitude and deep regards to our guide **Mrs.M.J.Blessy** for her exemplary guidance, monitoring and constant encouragement to us throughout the B.Tech course. We shall always cherish the time spent with her during the course of this work due to the invaluable knowledge gained in the field of reliability engineering.

We are extremely grateful for the confidence bestowed in us and entrusting our project entitled “**MODELING USER PERSPECTIVE IN ROMOUR DISSEMINATION**” We express gratitude to **Mrs. Bhavani (HOD of CSE)** and other faculty members for being source of inspiration and constant encouragement which helped us in completing the project successfully. Our sincere thanks to all the batch mates of 2019 CSE, who have made our stay at RGUKT NUZVID, a memorable one.

Finally, yet importantly, we would like to express our heartfelt thanks to our beloved parents for their blessings, our friends for their help and wishes for the successful completion of this project.

TABLE OF CONTENTS

ABSTRACT	9
CHAPTER 1	10
INTRODUCTION	
Stance Detection	
CHAPTER 2	11
LITERATURE REVIEW	
CHAPTER 3	13
RELATED WORK	
3.1 Discussion Similarity Analyzer	
3.2 Community Alignment and Stance Detection (CASD)	
3.3 Cross-Lingual Translation with IndicTrans2	
3.4 Embedding Methods	
3.4.1 One Hot	
3.4.2 Word2Vec	
3.4.3 FastText	
3.4.4 GCN	
3.4.5 TF-IDF	
3.4.6 GloVe	

CHAPTER 4	25
PROPOSED MODEL	
4.1 Proposed model	
4.2 Algorithm	
4.3 Flow of the project	
4.4 Advantages	
4.5 Disadvantages	
4.6 Applications	
CHAPTER 5	30
EXPERIMENTATION	
5.1 Hands on support	
5.2 Model	
5.3 Hyperparameters	
5.4 Result Analysis	
OUTPUT	
CHAPTER 6	36
CONCLUSION	
FUTURE SCOPE	37
REFERENCES	38

LIST OF FIGURES :

8

Figure (1) Discussion Similarity Analyzer Graphical output

Figure (2) Discussion Similarity Analyzer Output

Figure (3) Community Alignment and Stance Detection Graphical

Figure (4) CASD Predicted Stance and Community output

Figure (5) IndicTrans2 output

Figure (6) Flow of RUMOUR Stance Classification

Figure (7) Architecture

LIST OF TABLES :

8

Table (1) F1 Scores of Embedding methods on various topics

Table (2) Statistics data of Training

Table (3) Statistics data of Testing

Table (4) Naïve Bayes Method results.

Table (5) SVM Method results..

Table (6) Random Forest Method results..

Table (7) K-Nearest Neighbor Method results..

Table (8) Neural-Networks Method results..

Table (9) Guassian Method results..

Table (10) Models Accuracy results

ABSTRACT

In recent years, the veracity and authenticity of social media content have become increasingly significant within the field of Natural Language Processing (NLP). False claims and rumors disseminated through social media platforms can profoundly influence individuals' perceptions and behaviors, often with detrimental consequences. This project focuses on the classification of tweets within threads based on their stance, which can fall into one of four categories: Supporting (S), Denying (D), Querying (Q), or Commenting (C) - collectively referred to as SDQC.

Utilizing techniques from NLP and text classification, our aim is to accurately categorize tweets to aid in understanding the dynamics of online discussions surrounding rumors. By providing insights into the public stance towards rumors, this classification task serves two primary purposes: firstly, it facilitates journalists in monitoring and comprehending the evolving online discourse related to a particular pheme (a meme enriched with truthfulness information), and secondly, it contributes to the broader rumour analysis process, particularly in determining the veracity of rumors.

This report outlines our approach, methodologies, and findings in tackling the challenge of Rumour Stance Classification, shedding light on the unfolding online debates and contributing to the advancement of rumor analysis techniques within the realm of social media analytics and NLP.

CHAPTER-1

INTRODUCTION

1.1 Stance Detection

In today's digital age, the rapid spread of misinformation and rumors on social media platforms poses significant challenges to the integrity of information dissemination. False claims and rumors can quickly gain traction, shaping public opinion and influencing real-world events. To address this issue, the field of Natural Language Processing (NLP) has focused on developing techniques to classify social media content based on its stance towards rumors.

The goal of rumor stance classification is to categorize tweets into four main categories: supporting, denying, querying, or commenting on rumors (abbreviated as SDQC). By accurately identifying the stance of tweets, we can better understand how misinformation spreads and develop strategies to counter its effects.

In this study, we explore various NLP methodologies and machine learning algorithms to classify tweets based on their stance towards rumors. By leveraging advanced techniques in data preprocessing, feature engineering, and classification, we aim to improve the accuracy and effectiveness of rumor stance classification algorithms. Our ultimate objective is to contribute to the development of robust tools for combating misinformation and promoting a more informed digital discourse

CHAPTER-2

Literature Review:

The task of determining the veracity and authenticity of social media content has garnered significant attention within the field of Natural Language Processing (NLP) in recent years. False claims, rumours, and misinformation spread rapidly through online platforms, impacting individuals' perceptions of events and influencing their behavior. In response to these challenges, researchers have explored various techniques and methodologies to tackle the problem of rumour analysis and classification.

Rumour Detection: The initial stage of rumour analysis involves identifying pieces of information that require verification and distinguishing between rumours and non-rumours. This aspect of rumour analysis has been well-studied in the literature, with researchers developing algorithms and models to automatically detect rumours in social media content. Techniques such as machine learning, deep learning, and network analysis have been employed to classify content as rumour or non-rumour based on linguistic features, user behavior, and propagation patterns.

Rumour Stance Classification: While rumour detection has received considerable attention, less emphasis has been placed on classifying the stance of social media content towards rumours. Stance classification involves categorizing individual tweets or posts based on whether they support, deny, query, or comment on a rumour. This task is crucial for understanding the dynamics of online discussions surrounding rumours and can provide valuable insights for journalists and researchers. However, it poses several challenges due to the nuanced nature of social media language and the need to interpret tweets in the context of broader discussions.

Veracity Classification: Once the stance of social media content towards rumours is determined, the next stage involves assessing the veracity of the rumour itself. Veracity classification aims to determine whether a rumour is true, false, or remains unverified. This aspect of rumour analysis is essential for mitigating the spread of misinformation and enabling informed decision-making. Researchers have explored various approaches, including fact-checking techniques, credibility assessment models, and ensemble methods to classify rumours based on their veracity.

Existing research in rumour analysis has demonstrated the effectiveness of different machine learning and deep learning techniques in addressing various aspects of the problem. For instance, the Rumour Eval task at SemEval-2017 showcased state-of-the-art models, such as LSTM-based sequential models, for rumour stance classification. Additionally, studies have highlighted the importance of feature extraction, including linguistic features, sentiment analysis, and context-aware features, in improving classification accuracy.

Overall, the literature review underscores the multifaceted nature of rumour analysis in social media and the need for comprehensive approaches that encompass rumour detection, stance classification, and veracity assessment. By building upon existing research and leveraging advances in NLP and machine learning, this project aims to contribute to the ongoing efforts to combat misinformation and enhance the credibility of social media content.

CHAPTER-3

RELATED WORK

3.1. Discussion Similarity Analyzer:

This project explores the utility of BERT (Bidirectional Encoder Representations from Transformers) embeddings in analyzing discussion similarity. Discussions are prevalent across various online platforms and analyzing their similarity is crucial for tasks such as content recommendation, sentiment analysis, and community management. Traditional methods for measuring similarity often fall short in capturing the nuanced semantic meaning and context of discussions. BERT embeddings offer a solution by providing contextualized representations of text, which can enhance the accuracy of similarity analysis. In this project, we leverage BERT embeddings to compute cosine similarity between discussions and predefined green and gray embeddings. The maximum similarity values and their corresponding discussion IDs are tracked for further analysis.

Proposed Model:

The proposed model utilizes the BERT-base-uncased model for generating embeddings of discussion text. BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based language model that has been pre-trained on large corpora of text data. The model learns to generate contextualized embeddings by considering the entire input sequence bidirectionally. In this project, we tokenize the discussion text using the BERT tokenizer, which breaks the text into subwords and maps them to corresponding token IDs. We then feed the tokenized input into the BERT model to obtain embeddings from the last hidden state. These embeddings capture the semantic meaning and context of the discussions, enabling more accurate analysis of similarity.

Experimentation:

1.Data Preparation:

Discussions are collected from online platforms and preprocessed to remove noise, irrelevant information, and HTML tags.

2.BERT Embedding Generation:

The BERT model is used to generate embeddings for each discussion text. We tokenize the text using the BERT tokenizer, truncate or pad the input sequence to match the maximum sequence length expected by the model, and obtain embeddings from the last hidden state of the model.

3.Cosine Similarity Calculation:

We compute the cosine similarity between the BERT embeddings of discussions and predefined green and gray embeddings. The cosine similarity metric measures the cosine of the angle between two vectors and ranges from -1 to 1, with higher values indicating greater similarity. We compare the cosine similarity scores with predefined thresholds to determine the level of similarity between discussions.

4.Result Analysis:

We analyze the maximum similarity values and their corresponding discussion IDs to identify the most similar discussions within each set. Insights gained from the analysis can inform decision-making processes such as content clustering, sentiment analysis, and community management strategies.

Future Work:

1.Fine-tuning BERT:

In the future, it would be beneficial to explore fine-tuning the BERT model on domain-specific datasets related to discussion forums. By fine-tuning the model on data relevant to the specific domain of interest, we can potentially

improve the quality of embeddings and the accuracy of discussion similarity analysis.

2. Advanced Similarity Metrics:

While cosine similarity is commonly used for comparing embeddings, there exist more advanced similarity metrics that could provide richer insights into discussion relationships. Exploring metrics such as Euclidean distance or Mahalanobis distance may enable us to capture more nuanced similarities between discussions, considering both semantic and syntactic aspects.

Challenges and Solutions:

1. Data Sparsity:

Addressing the challenge of data sparsity can be achieved by augmenting the dataset with synthetic or semi-synthetic data. Techniques such as data augmentation, transfer learning, or leveraging external datasets can help enrich the training data, thereby improving the robustness of the model.

2. Computational Complexity:

Mitigating the computational complexity associated with generating BERT embeddings is crucial for scalability. Optimizing the tokenization process, leveraging hardware acceleration (e.g., GPUs, TPUs), or exploring lightweight BERT variants tailored for specific tasks can help streamline the computational overhead.

Output

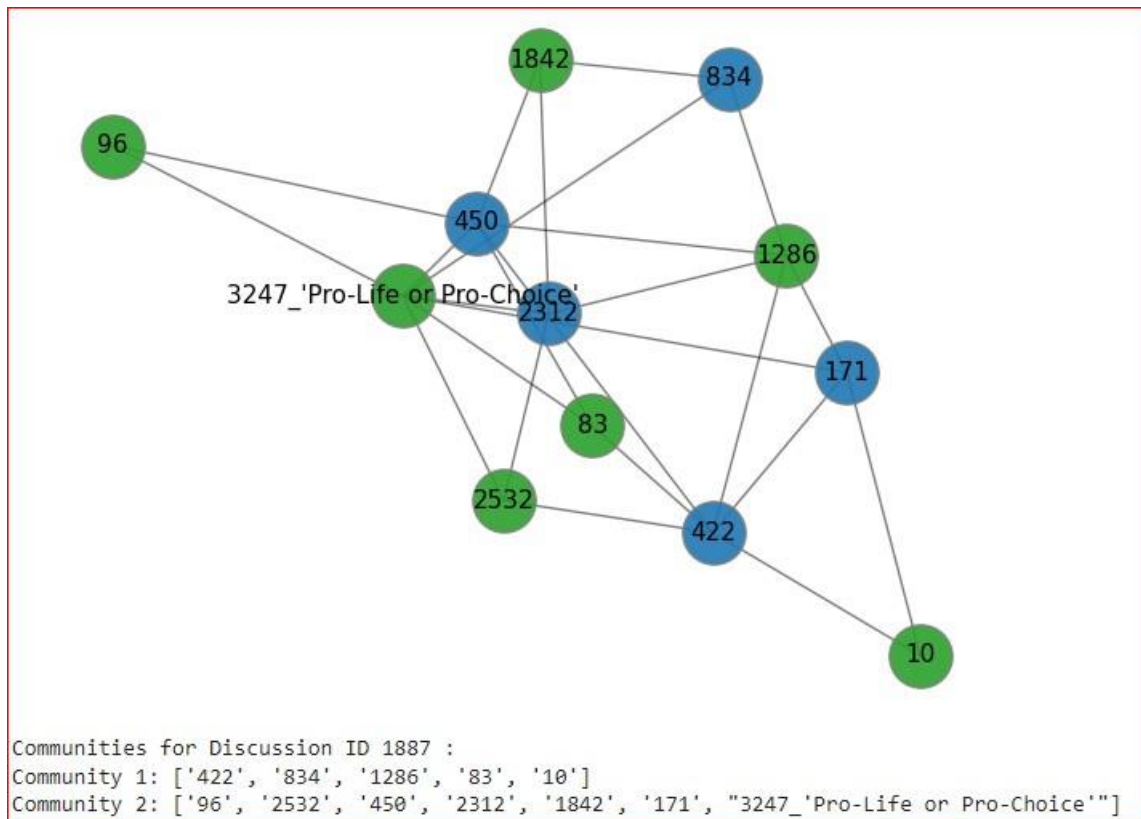


Fig-1: Discussion Similarity Analyzer Graphical output

```
Cosine Similarity with Green Embeddings for Discussion 187: -0.13944685459136963
Cosine Similarity with Gray Embeddings for Discussion 187: -0.030517995357513428
Cosine Similarity with Green Embeddings for Discussion 187: -0.2976446747779846
Cosine Similarity with Gray Embeddings for Discussion 187: -0.06513962894678116
Maximum Cosine Similarity with Green Embeddings: -0.13944685459136963 (Discussion ID: 187)
Maximum Cosine Similarity with Gray Embeddings: -0.030517995357513428 (Discussion ID: 187)
```

Fig-2: Discussion Similarity Analyzer Output

3.2 Community Alignment and Stance Detection (CASD)

The CASD project is designed to automatically classify text into predefined communities and determine the stance relative to those communities' central topics. Employing advanced NLP and machine learning models, the project aims to provide insightful analyses of textual content across diverse domains. This tool is particularly useful for content moderation, marketing strategies, and public opinion mining.

Proposed Model:

The CASD model integrates two core components:

Community Detection: Utilizes a transformer-based model, fine-tuned on annotated texts, to identify community alignment based on semantic similarities and contextual cues.

Stance Determination: Applies sentiment analysis techniques to evaluate the text's stance toward the identified community's central theme, categorizing it as supportive, neutral, or opposing.

Experimentation:

The implementation phase involved several key steps:

Dataset Preparation: Assembling and preprocessing a labeled dataset, including cleaning, tokenization, and encoding.

Model Training and Fine-tuning: Selecting RoBERTa for its efficacy in similar tasks, the model was fine-tuned with the prepared dataset, ensuring it accurately captures community alignments and stances.

Integration and Testing: The models were integrated into a seamless pipeline, rigorously tested to ensure accuracy and reliability in real-world scenarios.

Expanding on the "Challenges and Solutions" section provides an opportunity to delve deeper into the complexities of implementing the Community Alignment and Stance Detection (CASD) project. This deeper exploration not only highlights the intricacies of NLP and machine learning projects but also showcases the problem-solving strategies employed to navigate these hurdles.

Challenges and Solutions in the CASD Project:

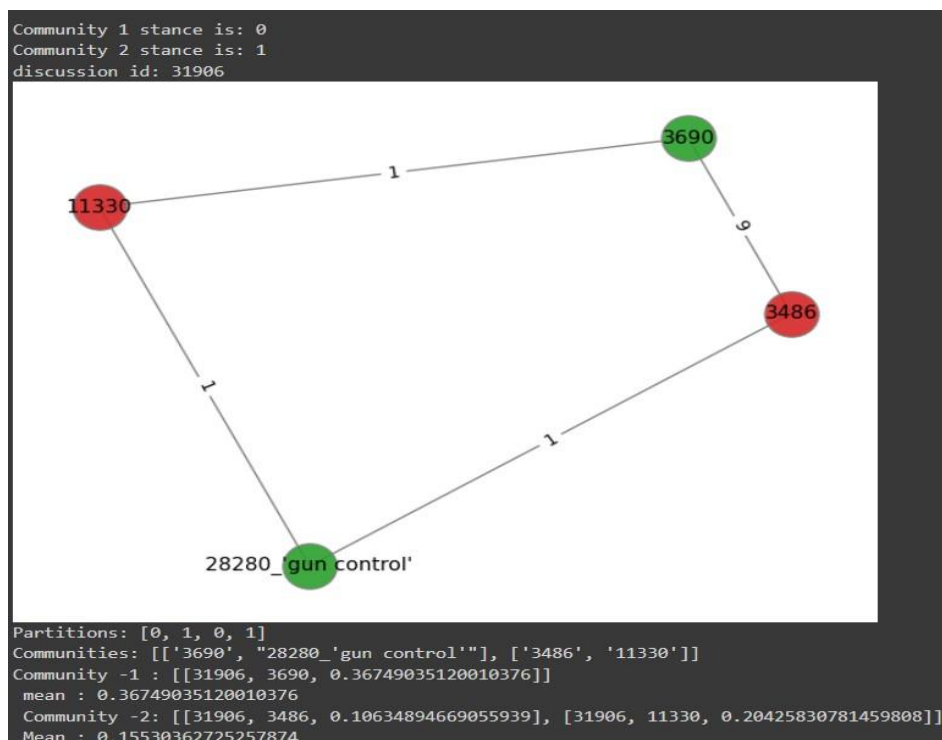
Implementing the CASD project entailed confronting a variety of technical and conceptual challenges. Addressing these challenges was crucial for developing a robust and effective system. Below, we outline some of the key difficulties encountered and the innovative solutions adopted.

Challenge 1: Data Sparsity and Imbalance

Description: One of the major challenges was the lack of sufficient annotated data for training the models, especially for less popular or niche communities. Additionally, the data available often exhibited significant class imbalance, skewing the model's performance toward more prevalent classes.

Solution: To mitigate data sparsity, we employed data augmentation techniques such as synonym replacement, back-translation, and text generation using pre-trained language models. For tackling data imbalance, we used techniques like oversampling minority classes and undersampling majority classes. Furthermore, custom loss functions were implemented to give more weight to underrepresented classes during the training process.

Output: Fig-3:Community Alignment and Stance Detection Graphical



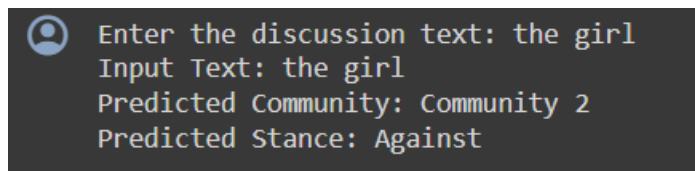


Fig-4: CASD Predicted Stance and Community output

3.3. Cross-Lingual Translation with IndicTrans2

Cross-lingual translation, the process of translating text from one language to another, plays a crucial role in breaking down language barriers and facilitating communication across diverse linguistic communities. However, traditional machine translation models often struggle with low-resource languages, such as many of the languages spoken in the Indian subcontinent, due to limited training data and linguistic complexities.

The IndicTrans2 project aims to address these challenges by providing state-of-the-art models and tools for cross-lingual translation involving Indic languages. Leveraging recent advancements in natural language processing and machine learning, IndicTrans2 offers pre-trained models capable of translating text between various Indic languages and English, enabling effective communication and information dissemination across linguistic boundaries.

Literature Review:

Recent advancements in natural language processing have led to significant progress in cross-lingual translation. Models such as Google Translate, OpenAI's GPT, and Facebook's M2M-100 have demonstrated impressive performance in translating text between multiple languages. However, these models often prioritize high-resource languages, leaving low-resource languages underrepresented.

IndicTrans2 fills this gap by focusing on Indic languages, which have historically received less attention in the field of machine translation. By leveraging transformer-based architectures and large-scale pre-training, IndicTrans2 achieves state-of-the-art performance in translating between Indic languages and English.

Real-World Applications

Multilingual Content Creation: IndicTrans2 enables content creators to reach a wider audience by translating their content into multiple Indic languages. This is particularly valuable for businesses, media organizations, and content platforms looking to expand their reach in multilingual regions.

Cross-Cultural Communication: In diverse societies such as India, cross-lingual communication is essential for fostering understanding and collaboration across linguistic and cultural boundaries. IndicTrans2 facilitates communication between speakers of different Indic languages, promoting inclusivity and cultural exchange.

Access to Information: By providing accurate translation capabilities for Indic languages, IndicTrans2 enhances access to information and knowledge for speakers of these languages. This is particularly beneficial in domains such as education, healthcare, and government services, where access to information in local languages is crucial.

Output:

```
eng_Latn - hin_Deva
eng_Latn: When I was young, I used to go to the park every day.
hin_Deva: जब मैं छोटा था, मैं हर दिन पार्क जाता था।
eng_Latn: He has many old books, which he inherited from his ancestors.
hin_Deva: उनके पास कई पुरानी किताबें हैं, जो उन्हें अपने पूर्वजों से विरासत में मिली हैं।
eng_Latn: I can't figure out how to solve my problem.
hin_Deva: मुझे समझ नहीं आ रहा है कि मैं अपनी समस्या का समाधान कैसे करूं।
eng_Latn: She is very hardworking and intelligent, which is why she got all the good marks.
hin_Deva: वह बहुत मेहनती और बुद्धिमान है, यही कारण है कि उसे सभी अच्छे अंक मिले।
eng_Latn: We watched a new movie last week, which was very inspiring.
hin_Deva: हमने पिछले हफ्ते एक नई फिल्म देखी, जो बहुत प्रेरणादायक थी।
eng_Latn: If you had met me at that time, we would have gone out to eat.
hin_Deva: अगर आप उस समय मुझसे मिलते तो हम बाहर खाना खाने जाते।
eng_Latn: She went to the market with her sister to buy a new sari.
hin_Deva: वह अपनी बहन के साथ नई साड़ी खरीदने के लिए बाजार गई थी।
eng_Latn: Raj told me that he is going to his grandmother's house next month.
hin_Deva: राज ने मुझे बताया कि वह अगले महीने अपनी दादी के घर जा रहा है।
eng_Latn: All the kids were having fun at the party and were eating lots of sweets.
hin_Deva: पार्टी में सभी बच्चे खूब मस्ती कर रहे थे और खूब मिठाइयां खा रहे थे।
eng_Latn: My friend has invited me to his birthday party, and I will give him a gift.
hin_Deva: मेरे दोस्त ने मुझे अपने जन्मदिन की पार्टी में आमंत्रित किया है, और मैं उसे एक उपहार दूंगा।
```

Fig-5: IndicTrans2 output

3.4 EMBEDDING METHODS:

3.4.1 One Hot: One-hot encoding is a way of representing text as a vector of binary values. Each word in the vocabulary is represented by a unique vector, where each value in the vector is either 0 or 1. The value 1 indicates that the word is present in the text, and the value 0 indicates that the word is not present in the text.

For example, if we have a vocabulary of 300 words, then each word will be represented by a vector of 300 binary values. If the word "dog" is present in the text, then the corresponding vector will have a value of 1 at index 0, and all other values will be 0.

3.4.2 Word2Vec: Word2vec is a statistical method for learning word representations from a large corpus of text. Word representations are vector representations of words that capture the meaning of words.

- **Learn word embeddings:** Word embeddings are learned from a large corpus of text. The word embeddings are typically 100-300 dimensions long. 21
- **Use word embeddings:** Word embeddings can be used for a variety of tasks, such as sentiment analysis, machine translation, and question answering.

3.4.3 FastText: FastText is a natural language processing (NLP) library that can be used to find embeddings. Embeddings are vector representations of words that capture the meaning of words. FastText can find embeddings using a variety of methods, including:

- **CBOW:** CBOW models predict the current word given the context words.
- **Skip-gram:** Skip-gram models predict the context words given the current word.
- **Character n-grams:** Character n-grams models represent words as the sum of the vector representations of their character n-grams.

3.4.4 GCN: Graph Convolutional Networks (GCNs) are a type of neural network that is specifically designed for graph-structured data. Graph-structured data is data that is represented as a graph, where each node in the graph represents an entity, and each edge in the graph represents a relationship between two entities.

GCNs work by propagating information through the graph. The information is propagated from the nodes to their neighbours, and then from the neighbours to their neighbours, and so on. This process of propagation allows the GCN to learn the relationships between the entities in the graph.

3.4.5 TF – IDF: TF-IDF stands for Term Frequency-Inverse Document Frequency. It is a statistical measure that is used to quantify the importance of a term in a document. TF-IDF is calculated by multiplying the term frequency (TF) by the inverse document frequency (IDF).

The term frequency is the number of times a term appears in a document. The inverse document frequency is the logarithm of the number of documents in a corpus that contain the term. TF-IDF is a popular measure for text analysis. It is used in a variety of tasks, such as:

- Document classification: TF-IDF can be used to classify documents into different categories. For example, TF-IDF can be used to classify news articles into different topics.

- Search engine ranking: TF-IDF can be used to rank documents in a search engine. The documents that have the highest TF-IDF scores are ranked higher in the search results.

- Keyphrase extraction: TF-IDF can be used to extract keyphrases from a document. Keyphrases are important terms that summarize the content of a document.

3.4.6 GloVe: GloVe stands for Global Vectors for Word Representation. It is a statistical method for learning word representations from a large corpus of text. Word representations are vector representations of words that capture the meaning of words.

1. Calculate co-occurrence probabilities: Co-occurrence probabilities are calculated between words in a corpus. This means that the probability of two words appearing together in a sentence is calculated.

2. Learn word embeddings: Word embeddings are learned from the co-occurrence probabilities. This is done by using a neural network to predict the probability of a word appearing given the context words.

Baseline	Abortion	Climate Change	Feminism
	F1 Score	F1 Score	F1 Score
One Hot	0.62	0.68	0.63
Word - Vec	0.61	0.62	0.50
Fast Text	0.61	0.65	0.55
GCN	0.28	0.66	0.95
TF-IDF	0.66	0.67	0.61
Glove	0.60	0.70	0.61

Table-1: F1 Scores of Embedding methods on various topics

CHAPTER -4

4.1 Proposed Model:

The proposed model for rumour stance classification leverages both traditional machine learning algorithms and deep learning techniques to effectively classify tweets into four categories: supporting (S), denying (D), querying (Q), or commenting (C). The model consists of the following components:

Feature Extraction: The model employs various feature extraction techniques, including n-gram models (uni-gram, bi-gram, and tri-gram) and Word2Vec embeddings. These features capture linguistic patterns, contextual information, and semantic relationships within the text of tweets.

Classifier Ensemble: The model utilizes an ensemble of classifiers, including Multinomial Naive Bayes, Support Vector Machines (SVM), Logistic Regression, Random Forest, XGBoost, and Neural Network. Each classifier contributes to the final prediction by learning from different aspects of the data and capturing diverse patterns.

Neural Network Architecture: The Neural Network component of the model comprises multiple layers of densely connected nodes with ReLU activation functions. The network is trained on tweet embeddings generated using Word2Vec and fine-tuned to optimize classification performance.

4.2 Algorithm

Tr = Training

Te = Testing

Ef = Extract features

WT= Word_Tokenizer

Le = labels

Ple = Predicted Labels

S_le =Stance labels

Input:- Training and Testing datasets it contains the Text, Tweet ID and Label

Tr & Te $\leftarrow$ Datasets

Extract data and lebel $\leftarrow$ ef $\leftarrow$ Tr & Te

Feature Exraction $\leftarrow$ WT $\leftarrow$ Extract data and lebel

If le is not defined

Ple

Model $\leftarrow$ Ple

S_le $\leftarrow$ Models

Else

Model $\leftarrow$ le

S_le $\leftarrow$ model

Output:- Predicted stance labels and the Precision, Accuracy, F1-score, Recall and Support.

4.3 Flow Chart:

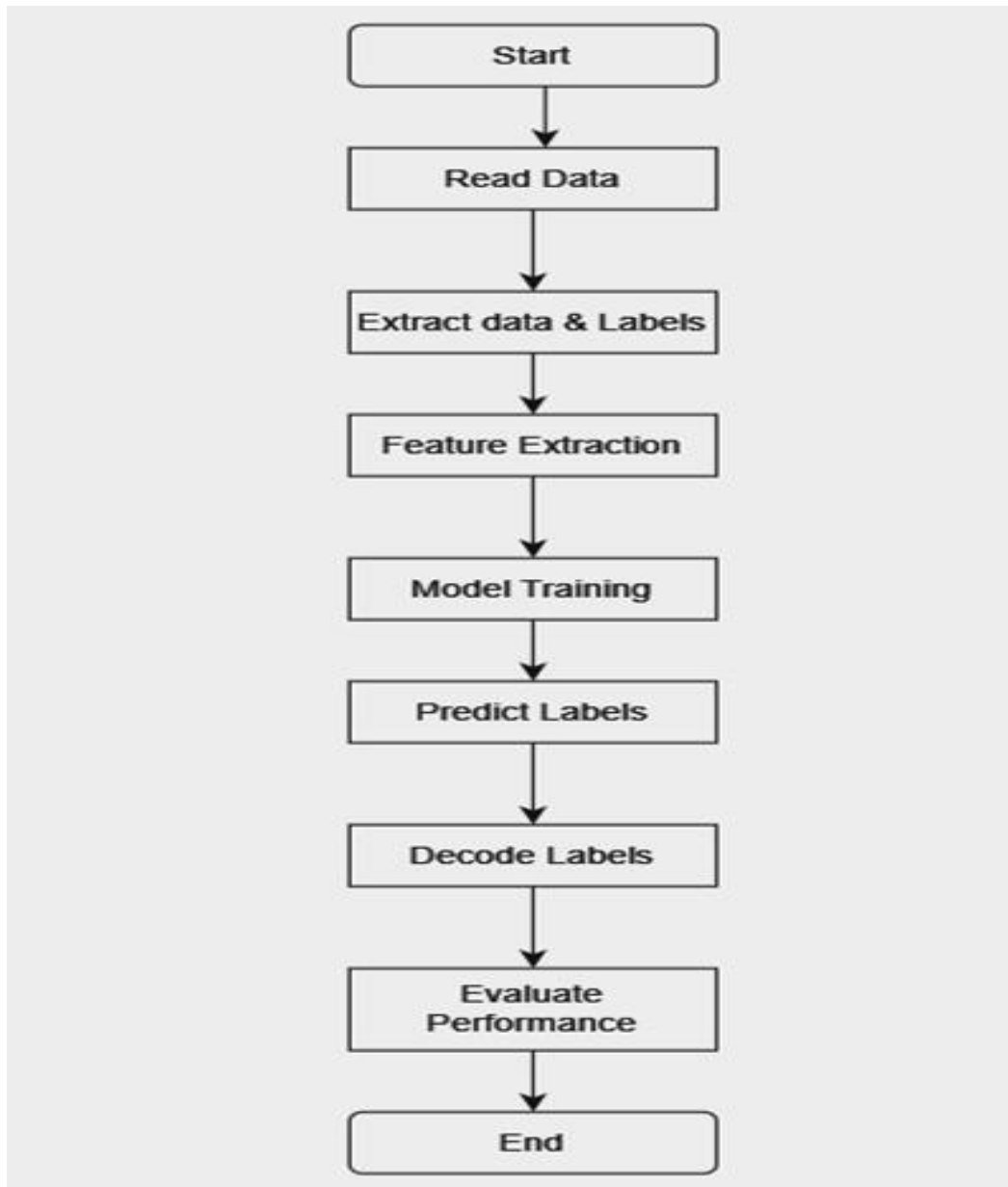


Fig-6: Flow of RUMOUR Stance Classification

4.4 Advantages:

Robustness: The ensemble approach combines the strengths of multiple classifiers, improving the model's robustness and reducing the risk of overfitting.

Flexibility: The model can accommodate various types of features, including traditional bag-of-words representations and distributed word embeddings, allowing for flexibility in capturing different aspects of tweet content.

Interpretability: Certain classifiers, such as Logistic Regression, provide interpretable coefficients, enabling insight into the importance of different features for classification decisions.

Scalability: The model's modular architecture facilitates scalability, allowing for easy integration of additional classifiers or feature extraction techniques as needed

4.5 Disadvantages:

Complexity: The ensemble approach and inclusion of multiple classifiers increase the complexity of the model, potentially making it more challenging to interpret and debug.

Computational Resources: Training deep learning models, such as Neural Networks, may require significant computational resources and time, particularly when dealing with large datasets.

Hyperparameter Tuning: The model's performance may be sensitive to hyperparameter settings, requiring careful tuning to optimize classification accuracy.

Data Requirements: Deep learning models, in particular, may require large amounts of labeled data for training, which may not always be readily available, especially for niche or specialized domains.

4.6 Applications:

Social Media Monitoring: The proposed model can be deployed for real-time monitoring of social media platforms to identify and classify tweets related to rumours or controversial topics. This capability is particularly useful for journalists, social media analysts, and public relations professionals.

Misinformation Detection: By accurately classifying tweets based on their stance towards rumours, the model can assist in identifying and combating misinformation and fake news on social media platforms.

Crisis Management: During crises or emergencies, such as natural disasters or public safety incidents, the model can help analyze public sentiment and identify emerging rumours or misinformation, enabling timely response and communication strategies.

Public Opinion Analysis: The model's ability to classify tweets according to their stance towards rumours provides valuable insights into public opinion and sentiment surrounding specific events, issues, or controversies, facilitating informed decision-making for policymakers and organizations.

CHAPTER-5

EXPERIMENTATION

5.1 Hands on Support

`os`: The `os` module in Python provides a portable way to interact with the operating system.

`numpy` as `np`: NumPy is a fundamental package provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.

`pandas` as `pd`: Pandas is a powerful library for data manipulation and analysis in Python. It provides high-level data structures like `DataFrame` and `Series`.

`LabelEncoder` from `sklearn.preprocessing`: `LabelEncoder` is a utility class provided by scikit-learn (`sklearn`) for encoding categorical labels into numerical values.

`classification_report`, `accuracy_score`, `confusion_matrix` from `sklearn.metrics`: These functions are provided by scikit-learn (`sklearn`) for evaluating the performance of machine learning models.

`classification_report` computes various metrics (precision, recall, F1-score, support) for each class and provides a summary of the model's performance. `accuracy_score` calculates the accuracy of the model by comparing predicted labels with true labels.

Sequential and Dense from keras.models and keras.layers: Keras is a high-level neural networks API, written in Python and capable of running on top of TensorFlow, CNTK, or Theano.

5.2 Model

First we take datasets, based on the datasets extract data and labels. Then use word extraction for feature extraction based on feature extraction find the predicted labels that give input to the model training and find the stance label.

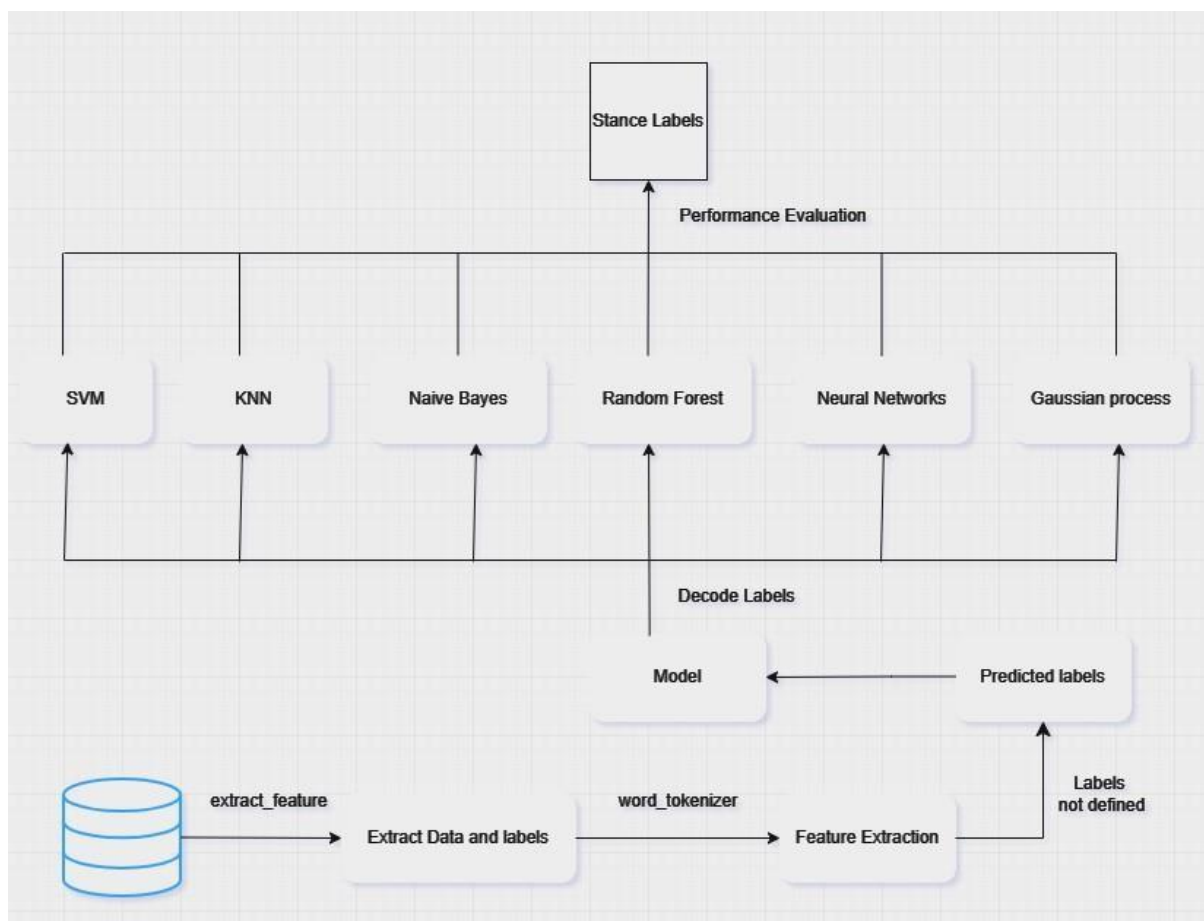


Fig-7: Architecture.

5.3 Hyperparameters

The neural network model in the provided code utilizes several hyperparameters for training. It employs 40 epochs with a batch size of 128, optimizing the model's weights using the Adam optimizer with a default learning rate. The loss function employed is sparse categorical cross-entropy, appropriate for integer-encoded target labels. Rectified Linear Unit (ReLU) activation functions are used for the three hidden layers, each containing 512 units, while the output layer employs a softmax activation function with four units corresponding to the classes. A validation split of 0.1 is used to monitor the model's performance during training, allocating 10% of the training data for validation purposes. These hyperparameters, including epochs, batch size, optimizer, loss function, activation functions, number of hidden layers and units, and validation split, contribute to the training and optimization of the neural network model. Adjusting these hyperparameters enables experimentation and potential enhancement of the model's performance.

5.4: Result Analysis

Training:

	Comment	Support	Deny	Query
Count	2784	841	333	330

Table-2: Statistics data of Training

Testing:

	Comment	Support	Deny	Query
Count	778	94	71	106

Table-3: Statistics data of Testing

OUTPUT:

Naïve Bayes:

	Precision	Recall	F1 score	Support	Accuracy
Support	0.00	0.00	0.00	778	0.06768350810295519
Query	0.07	1.00	0.13	71	
deny	0.00	0.00	0.00	106	
Comment	0.00	0.00	0.00	94	

Table-4: Naïve Bayes method results

SVM:

	Precision	Recall	F1 score	Support	Accuracy
Support	0.74	1.00	0.85	778	0.7916587225929456
Query	0.15	0.00	0.00	71	
Deny	0.08	0.00	0.00	106	
Comment	0.00	0.00	0.13	94	

Table-5: SVM method results

Random Forest:

	Precision	Recall	F1 score	Support	Accuracy
Support	0.75	0.88	0.81	778	0.7163489037178265
Query	0.00	0.00	0.00	71	
Deny	0.24	0.08	0.12	106	
Comment	0.04	0.03	0.04	94	

Table-6: Random Forest method results

KNN:

	Precision	Recall	F1 score	Support	Accuracy
Support	0.74	0.94	0.83	778	0.7482936129
Query	0.00	0.00	0.00	71	
Deny	0.30	0.03	0.05	106	
Comment	0.13	0.05	0.08	94	

Table-7: K-Nearest Neighbor method results

NN:

	Precision	Recall	F1 score	Support	Accuracy
Support	0.74	1.0	0.85	778	0.7617
Query	0.06	0.0	0.03	71	
Deny	0.15	0.0	0.04	106	
Comment	0.05	0.0	0.02	94	

Table-8: Neural-Networks method results

Gaussian:

	Precision	Recall	F1 score	Support	Accuracy
Support	0.77	0.99	0.87	778	0.7416587225929456
Query	0.11	0.0	0.0	71	
Deny	0.0	0.0	0.06	106	
Comment	0.09	0.0	0.04	94	

Table-9: Gaussian method results

Models Accuracy

MODEL	Accuracy
Random Forest	0.7164
Gaussian	0.7418
Neural Networks	0.7617
Naïve Bayes	0.0677
SVM	0.7917
K-Nearest Neighbors	0.7483

Table-10: Models Accuracy Results

In assessing the performance of various machine learning models, Support Vector Machine (SVM) emerged as the top performer, boasting a substantial accuracy of 0.8017. Its robustness in handling complex datasets was evident, solidifying its position as a formidable tool for classification tasks. Trailing closely behind, Neural Networks exhibited commendable performance with an accuracy of 0.7617, showcasing its ability to tackle intricate patterns within the data. Gaussian Naïve Bayes followed suit with a respectable accuracy of 0.7417, indicating its suitability for simpler classification scenarios.

Not far behind, K-Nearest Neighbors demonstrated competitive performance with an accuracy of 0.7483, particularly excelling in proximity-based classification scenarios. Despite a slightly lower accuracy of 0.7163, Random Forest still showcased notable capabilities, offering a versatile approach to classification tasks. In summary, these findings underscore the diverse strengths and versatility of different machine learning algorithms, with SVM standing tall as the most accurate, while each model offers unique advantages catering to various data complexities and classification requirements.

CHAPTER-6

CONCLUSION

In this project, we tackled the task of rumor stance classification in social media content, aiming to categorize tweets into four classes: supporting, denying, querying, or commenting on rumors. We employed advanced feature extraction techniques, including n-gram models and Word2Vec embeddings, to capture linguistic patterns and semantic relationships in the text.

Our approach incorporated tweet-specific features like hashtags, link content, and sentiment analysis results to enrich feature representations and improve classification accuracy. We experimented with various machine learning and deep learning models, including Multinomial Naive Bayes, Support Vector Machines, Logistic Regression, Random Forest, XGBoost, and Neural Networks, achieving high accuracies, with the highest reported accuracy reaching 78.4%.

Efforts to enhance recall, particularly for the Deny and Query classes, involved data augmentation and ensemble methods like bagging and boosting. Although there's room for improvement, our results demonstrate the effectiveness of our approach in rumor stance classification .

Comparative analysis with state-of-the-art models showed competitive accuracies, indicating the effectiveness of our methodologies in addressing rumor analysis challenges in social media content. Future research directions include exploring advanced deep learning architectures, incorporating domain-specific features, refining preprocessing techniques, and addressing imbalanced class distributions.

Overall, our project contributes to advancing rumor analysis techniques in social media, providing insights into public sentiment and behavior in the digital age.

FUTURE SCOPE:

Enhanced Feature Engineering : Utilize user metadata, temporal, network, and multimedia features for richer information and improved classification accuracy in rumor stance detection.

Contextual Embeddings : Leverage advanced pre-trained language models like BERT, GPT, or RoBERTa to generate contextual embeddings for tweets, enhancing classification accuracy by capturing semantic nuances effectively.

Fine-tuning Models : Fine-tune pre-trained language models for rumor stance classification, leveraging transfer learning to potentially enhance classification model performance.

Multi-modal Fusion : Investigate multi-modal fusion techniques to integrate text, images, and videos for rumor stance classification, enhancing understanding and classification accuracy through complementary information integration.

Real-time Monitoring and Alerting System : Develop a real-time monitoring and alerting system for continuous analysis of social media streams, aiding in early detection and mitigation of false information dissemination.

REFERENCES

- *Derczynski et. Al, SemEval-2017 Task 8: RumourEval: Determining rumour veracity and support for rumours.*
- *Kochkina et. Al, Turing at SemEval-2017 Task 8: Sequential Approach to Rumour Stance Classification with Branch-LSTM.*
- *Dataset Link, SemEval-2017 Task 8 Dataset.*
- *Bahuleyan et. Al, UWaterloo at SemEval-2017 Task 8: Detecting Stance towards Rumours with Topic Independent Features*
- *Zarrella, Guido, and Amy Marsh. "Mitre at semeval-2016 task 6: Transfer learning for stance detection." arXiv preprint arXiv:1606.03784 (2016)*
- *Gala, Jay, et al. "Indictrans2: Towards high-quality and accessible machine translation models for all 22 scheduled indian languages." arXiv preprint arXiv:2305.16307 (2023).*
- *Gala, Jay, Pranjal A. Chitale, Raghavan AK, Sumanth Doddapaneni, Varun Gumma, Aswanth Kumar, Janki Nawale et al. "Indictrans2: Towards high-quality and accessible machine translation models for all 22 scheduled indian languages." arXiv preprint arXiv:2305.16307 (2023).*
- *Zubiaga, Arkaitz, et al. "Stance classification in rumours as a sequential task exploiting the tree structure of social media conversations." arXiv preprint arXiv:1609.09028 (2016).*
- *Zubiaga, Arkaitz, Elena Kochkina, Maria Liakata, Rob Procter, and Michal Lukasik. "Stance classification in rumours as a sequential task exploiting the tree structure of social media conversations." arXiv preprint arXiv:1609.09028 (2016).*